**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**
**Department of Computer Science and Engineering**
**CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation**

| | | | |
|---|---|---|---|
| **Course Code:** | CSA10 / CSA1063 | **Course Title:** | Software Engineering |
| **CO Assessed:** | CO3 | **Assessment:** | Assessment Tool 1 – Code Review & Repository Evaluation |
| **Weightage:** | 20% | **Total Marks:** | 25 |
| **CO3 Statement:** | Build full-stack applications with an emphasis on containerization, version control, and collaborative development | | |
| **Student Name:** | Malli Chandana | **Reg. No.:** | 192472250 |
| **Date:** | | **Duration:** | 60 minutes |

**Code of Conduct**
*I Malli Chandana(192472250) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*
Signature of the Student: Malli Chandana

## Annexure A – Code Review & Repository Evaluation

### Rubrics for Calculation

| Evaluation Criteria | Excellent (4 Marks) | Good (3 Marks) | Satisfactory (2 Marks) | Needs Improvement (1 Mark) |
|---|---|---|---|---|
| **Problem Analysis & Repository Organization(15%)** | Clearly explains the problem, solution, and repository structure with logical organization and justification. | Good explanation with minor omissions. | Basic explanation and repository organization. | Incomplete or poorly organized repository. |
| **Code Quality Review(25%)** | Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations. | Good code review with minor gaps. | Basic review with limited analysis. | Superficial or inaccurate code review. |
| **Version Control Evaluation(20%)** | Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices. | Good evaluation with adequate explanation. | Basic evaluation with limited evidence. | Poor or incomplete version control analysis. |
| **Testing & Validation(15%)** | Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence. | Good testing with minor omissions. | Basic testing with limited evidence. | Inadequate or missing testing and validation. |
| **Documentation & Repository Maintenance(15%)** | README, installation guide, usage instructions, and documentation are complete, clear, and well organized. | Documentation is mostly complete. | Basic documentation with missing details. | Poor or insufficient documentation. |
| **Review Findings, Recommendations & Presentation(10%)** | Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references. | Good recommendations and presentation. | Basic recommendations with acceptable presentation. | Weak conclusions and poor presentation. |

## Code Review and Repository Evaluation:

# Smart Heritage Site Information and Virtual Tour Platform

### 1. Problem Overview

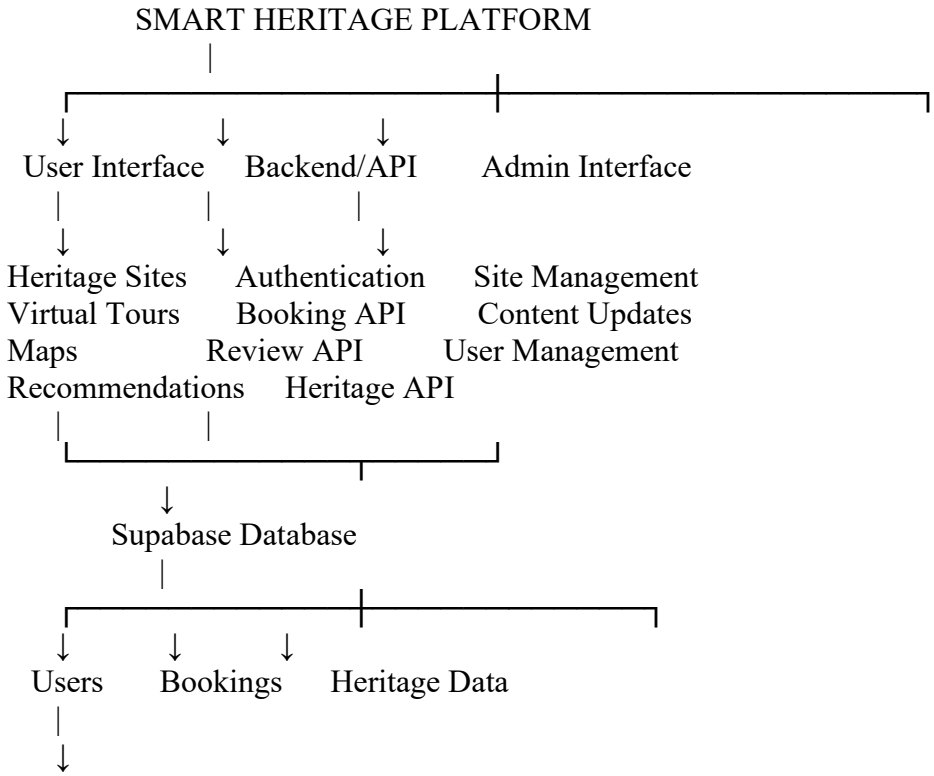
### 1.1 Problem Statement

Traditional heritage-site information systems generally provide only basic descriptions, photographs, location details, and visiting information. Visitors may need to search multiple sources to understand the historical importance, architecture, cultural significance, timings, ticket information, and available facilities of a heritage location.The **Smart Heritage Site Information and Virtual Tour Platform** addresses these limitations by providing a centralized digital platform where users can explore heritage sites, access detailed historical information, view images and 360° virtual tours, obtain location information, and interact with an intelligent heritage assistant.The platform combines web technologies, database services, maps, virtual-tour functionality, and AI-based assistance to provide a more interactive tourism experience.

### 1.2 Implemented Solution

The proposed platform consists of a frontend interface, backend services, database, AI assistance, map services, and virtual-tour components.

The major components are:



AI Assistant
|
↓
Gemini API / AI Service

### 1.3 Project Objectives

The main objectives are:

- To provide centralized information about heritage sites.
- To provide an interactive **360° virtual tour** experience.
- To provide AI-assisted information and guidance.
- To provide multilingual support.
- To provide personalized heritage-site recommendations.
- To provide online ticket-booking functionality.
- To provide digital maps and location information.
- To allow users to submit reviews and ratings.
- To provide administrators with facilities to manage heritage-site content.

### 1.4 Key Functionalities

The important functionalities of the platform include:

**User Registration and Login:** Users can create accounts and securely access the platform.

**Heritage Site Exploration:** Users can browse heritage sites and view historical information, images, descriptions, and other details.

**Virtual Tour:** Users can explore selected heritage sites through 360° panoramic content.

**AI Heritage Assistant:** Users can interact with an AI-based assistant to obtain information about heritage locations.

**Maps and Navigation:** Users can view heritage locations using an interactive map.

**Booking:** Users can book tickets or visits through the platform.

**Reviews:** Users can submit reviews and ratings for heritage sites.

**Admin Management:** Administrators can manage site information, images, bookings, and user-related content.

## 2. Repository Organization

A well-organized repository is important because it makes the project easier to understand, debug, modify, and maintain. For the Smart Heritage Platform, the repository should separate the frontend, backend, configuration, documentation, and static resources.

### 2.1 Recommended Repository Structure

```
smart-heritage-platform/
│
├── frontend/
│   ├── public/
│   └── src/
│       ├── components/
│       ├── pages/
│       ├── layouts/
│       ├── services/
│       ├── hooks/
│       ├── utils/
│       ├── assets/
│       └── App.jsx
│
├── backend/
│   ├── controllers/
│   ├── routes/
│   ├── models/
│   ├── middleware/
│   ├── services/
│   ├── config/
│   └── server.js
│
├── database/
│   ├── schema/
│   └── seed/
│
├── docs/
│   ├── architecture.md
│   ├── api-documentation.md
│   └── user-guide.md
│
├── tests/
│   ├── unit/
│   ├── integration/
│   └── api/
│
├── .env.example
├── .gitignore
├── README.md
├── package.json
└── LICENSE
```

frontend
backend
database
docs
tests
README.md
package.json
.gitignore

### 2.2 Folder Organization

The repository follows a modular organization where each major responsibility is separated into its own directory.

The **frontend** contains user-interface components, pages, assets, and client-side services.

The **backend** contains API routes, controllers, middleware, database-related services, and server configuration.

The **database** directory can contain database schema and sample data.

The **tests** directory separates testing code from application code.

The **docs** directory stores technical and user documentation.

This separation improves maintainability because developers can identify the location of a particular functionality without searching the entire project.

### 2.3 File Naming Conventions

Consistent naming conventions improve readability.

Examples:

HeritageCard.jsx
VirtualTour.jsx
BookingPage.jsx
LoginForm.jsx
heritageService.js
bookingController.js
authMiddleware.js

Files should have descriptive names rather than names such as:

file1.js
newcode.js
testfinal.js
latestfinal2.js

Descriptive names allow developers to understand the purpose of a file without opening it.

## 3. Code Quality Review

### 3.1 Readability

Readable code is important because multiple developers may work on the project.

A good implementation should use:

- Meaningful variable names
- Meaningful function names
- Consistent indentation
- Proper spacing
- Appropriate comments
- Small and focused functions

**Example of Readable Code**

```
const calculateBookingTotal = (ticketPrice, quantity) => {
   return ticketPrice * quantity;
};
```

The function name clearly describes its purpose.

A less readable implementation would be:

```
const x = (a, b) => a * b;
```

Although both implementations work, the first version is easier to understand and maintain.

### 3.2 Modularity

The platform contains several independent functionalities such as authentication, heritage information, booking, reviews, maps, and virtual tours.

These functionalities should be separated into reusable components.

For example:

```
components/
│
├── Navbar.jsx
├── HeritageCard.jsx
├── SearchBar.jsx
├── BookingForm.jsx
├── ReviewCard.jsx
├── MapView.jsx
└── VirtualTour.jsx
```

This approach prevents the entire application from being implemented in a single large file.

If the booking functionality needs modification, developers can work primarily with the booking-related components instead of changing unrelated sections.

### 3.3 Coding Standards

The project should follow consistent coding standards.

Recommended practices include:

- Use consistent indentation.
- Use meaningful names.
- Avoid duplicate code.
- Keep functions short.
- Separate UI and business logic.
- Handle errors properly.
- Avoid unnecessary comments.
- Remove unused variables and imports.
- Keep sensitive information outside the source code.

For example, API keys should **not** be directly written in source files.

Incorrect:

```
const API_KEY = "my-secret-api-key";
```

Instead, environment variables should be used:

```
const API_KEY = process.env.GEMINI_API_KEY;
```

### 3.4 Documentation Within Code

Important functions should contain short comments or documentation explaining their purpose.

Example:

```
// Retrieves heritage sites based on the selected category
async function getHeritageSites(category) {
    const response = await api.get(`/heritage?category=${category}`);
    return response.data;
}
```

Comments should explain **why** something is done when the reason is not obvious, rather than describing every simple line.

## 4. Code Review – Strengths and Areas for Improvement

### Strengths

The Smart Heritage Platform has several strengths from a software-engineering perspective.

### Modular Architecture

Separating frontend, backend, database, and services improves maintainability.

### Reusable Components

Reusable components such as heritage cards, navigation bars, booking forms, and review components reduce code duplication.

### API-Based Communication

Using APIs between frontend and backend provides a clear separation between presentation and business logic.

### Authentication

User authentication helps protect user-specific information and administrative functionality.

### External Service Integration

Integration with mapping, AI, storage, and virtual-tour services allows the application to provide advanced functionality without implementing every service from scratch.

### Areas for Improvement

The following improvements should be considered:

1. Add stronger input validation.
2. Improve error handling.
3. Remove unused code and dependencies.
4. Add automated unit tests.
5. Add API integration tests.
6. Use consistent naming conventions.
7. Add proper API documentation.
8. Protect environment variables.
9. Add loading and error states to UI components.
10. Improve accessibility of the user interface.

## 5. Version Control Evaluation

Git is important for managing changes to the Smart Heritage Platform.

A repository should maintain a clear history of development rather than storing only the final version.

### 5.1 Commit History

Good commit messages should describe a specific change.

**Good examples**

feat: add heritage site search
feat: implement virtual tour component
fix: resolve booking validation error
fix: correct map marker coordinates
docs: update installation instructions
test: add booking API tests

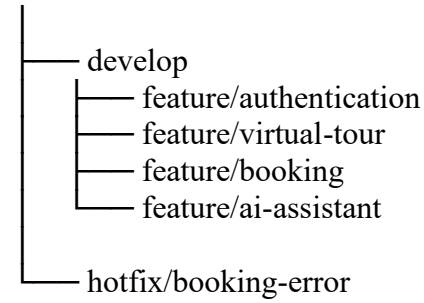
**Poor examples**

update
changes
final
final2
new
working

Good commit messages make it easier to understand the development history.

### 5.2 Branching Strategy

A simple branching structure can be used:




The main branch should contain stable code.

Feature branches can be used for individual functionalities.

After testing and review, feature branches can be merged into the development or main branch.

## 6. Code Testing and Validation

Testing ensures that the Smart Heritage Platform works according to its requirements.

Testing should be performed at multiple levels.

### 6.1 Unit Testing

Individual functions and components can be tested independently.

Examples:

- Login validation
- Booking calculation
- Search function
- Recommendation logic
- Review validation

Example:

Test: Booking Total Calculation

Input:
Ticket Price = ₹200
Quantity = 3

Expected:
₹600

Result:
₹600

Status:
PASS

### 6.2 API Testing

API endpoints can be tested using Postman or another API testing tool.

Important APIs include:

POST /api/auth/login
POST /api/auth/register

GET /api/heritage
GET /api/heritage/:id

POST /api/bookings
GET /api/bookings/:userId

POST /api/reviews
GET /api/reviews/:siteId

### 6.3 Integration Testing

Integration testing verifies communication between components.

For example:

User
↓
React Frontend
↓
Backend API
↓
Database
↓
Response
↓
Frontend

A booking integration test should verify that:

1. User selects a heritage site.
2. User selects ticket quantity.
3. Booking request is sent.
4. Backend validates the request.
5. Database stores the booking.
6. Confirmation is returned to the user.

## 7. Sample Test Cases

| Test Case | Input/Action | Expected Result | Status |
|---|---|---|---|
| TC01 | Valid user login | User successfully logged in | PASS |
| TC02 | Invalid password | Error message displayed | PASS |
| TC03 | Search heritage site | Matching sites displayed | PASS |
| TC04 | Open virtual tour | 360° tour loads | PASS |
| TC05 | Submit valid booking | Booking confirmed | PASS |
| TC06 | Invalid booking data | Validation error shown | PASS |
| TC07 | Submit review | Review stored/displayed | PASS |
| TC08 | Open map | Heritage location displayed | PASS |
| TC09 | AI assistant query | Relevant response displayed | PASS |
| TC10 | Admin updates site | Updated information displayed | PASS |

Replace the **PASS** values with your actual execution results if any test has not yet been run.

## 8. Repository Documentation

A good repository should contain a comprehensive README.md.

The README should include:

**Project Title**

**Smart Heritage Site Information and Virtual Tour Platform**

**Project Description**

A short explanation of the purpose and functionality of the system.

**Features**

• Heritage site information
• 360° virtual tours
• AI heritage assistant
• Maps and navigation
• Online booking
• Reviews and ratings
• Personalized recommendations
• Multilingual support
• Admin management

**Technology Stack**

Frontend:
React.js
HTML5
CSS3
JavaScript
Tailwind CSS

Backend:
Node.js
Express.js

Database:
Supabase PostgreSQL

AI:
Gemini API

Maps:
Leaflet + OpenStreetMap

Virtual Tour:
Pannellum / 360° viewer

Version Control:
Git + GitHub

**Installation Instructions**

Example:

git clone <repository-url>
cd smart-heritage-platform

npm install

npm run dev

The README should also explain how environment variables are configured.

## 9. Documentation Improvements

The repository documentation can be improved by adding:

- System architecture diagram
- Database schema
- API documentation
- Installation guide
- User guide
- Admin guide
- Screenshots
- Troubleshooting section
- Testing instructions
- Deployment instructions
- Contribution guidelines

A developer should ideally be able to clone the repository and understand how to run the project without requiring additional explanation from the original developer.

## 10. Code Review Findings and Recommendations

The overall review indicates that the Smart Heritage Platform has a modular structure and contains multiple independent functionalities. However, maintainability can be further improved through better testing, documentation, error handling, and version-control practices.

### Recommendation 1 – Improve Code Modularity

Separate large components into smaller reusable components.

### Recommendation 2 – Add Automated Testing

Unit and integration tests should be added to reduce regression errors.

### Recommendation 3 – Improve Documentation

The README should contain complete installation, configuration, usage, and deployment instructions.

### Recommendation 4 – Improve Git Practices

Use meaningful commit messages and feature branches.

**Recommendation 5 – Secure Configuration**

API keys, database credentials, and other secrets should be stored in environment variables rather than source files.

**Recommendation 6 – Improve Error Handling**

The frontend should display meaningful error messages when APIs, database operations, maps, or virtual tours fail.

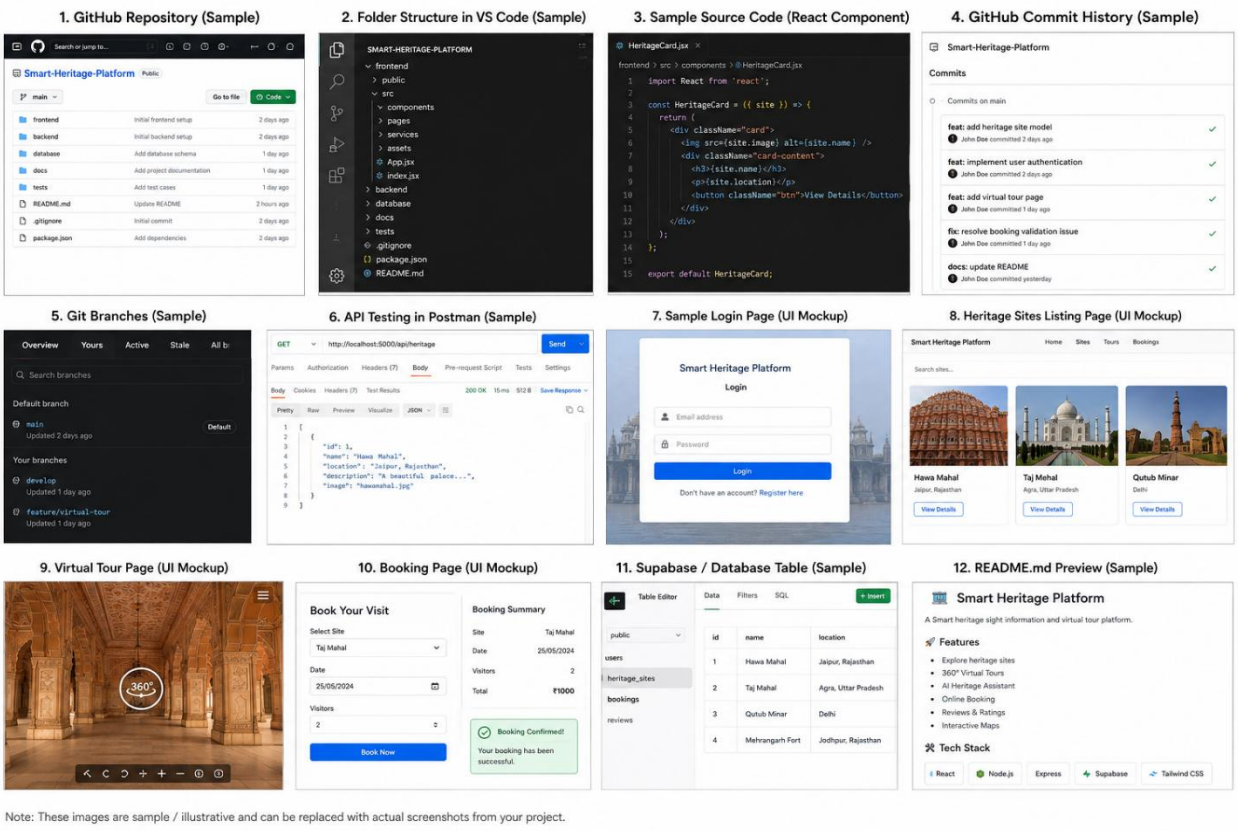
**Recommendation 7 – Add Continuous Integration**

A future version can use CI tools to automatically run tests whenever code is pushed to the repository.

## 11. Overall Repository Evaluation

The following evaluation can be used as the final summary:

| Evaluation Area | Assessment | Remarks |
|---|---|---|
| Repository Organization | Good | Separate frontend, backend, tests and documentation |
| Code Readability | Good | Meaningful names and consistent formatting should be maintained |
| Modularity | Good | Components and services can be separated by responsibility |
| Version Control | Good | Git supports history and collaboration |
| Testing | Needs Improvement | More automated tests should be added |
| Documentation | Good | README should include complete setup and usage details |
| Security | Needs Improvement | Secrets should use environment variables |
| Maintainability | Good | Modular architecture supports future enhancement |




## 12. Conclusion

The Smart Heritage Site Information and Virtual Tour Platform provides an integrated digital solution for exploring and interacting with heritage locations. The platform combines heritage information, virtual tours, AI assistance, maps, booking, reviews, and recommendation functionality.The code review shows that maintainability depends not only on the functionality of the application but also on how the source code and repository are organized. A modular folder structure, meaningful file names, reusable components, consistent coding standards, and proper error handling make the project easier to maintain.Git provides an effective mechanism for tracking changes and supporting collaborative development. Meaningful commits and feature-based branches can further improve the development workflow.Testing should cover individual functions, APIs, integration between frontend and backend, and complete user workflows. Similarly, a comprehensive README should provide installation, configuration, usage, testing, and deployment instructions.

Overall, the project provides a strong foundation for a smart tourism application. The major recommended improvements are increasing automated test coverage, improving repository documentation, strengthening error handling, following consistent Git practices, and securing configuration information. These improvements will make the platform more reliable, maintainable, scalable, and suitable for future development.

| Criteria | **Mark** |
|---|---|
| System Requirement Analysis **(30%)** | /5 |
| Selection of Software Architecture **(25%)** | /5 |
| Architecture Diagram and Component Design **(20%)** | /5 |
| Component Interaction and Quality Attribute Analysis **(15%)** | /5 |
| Architectural Justification and Technical Presentation **(10%)** | /5 |
| **TOTAL** | /25 |